

Simplicity covenant offers in the SeqOB

DEX: design pass

A design for funded, self-enforcing, non-interactive resting offers using Simplicity covenants, same-chain and on the Sequentia leg of cross-chain swaps. This is the design pass; the regtest experiment plan is at the end.

0. Framing (corrected)

Sequentia is transparent-by-default with confidentiality opt-in. A covenant can only police values it can read, and Simplicity introspection returns amounts/assets as `Either<blinded-point, explicit>` with no on-chain unblind jet, so covenant offers use EXPLICIT amounts and asset ids. On Sequentia that is the DEFAULT case, so there is no privacy tradeoff to weigh: covenant offers fit the default, and only the opt-in confidential path is mutually exclusive with covenant enforcement. The prior assessment's "confidential-by-default / transparency tradeoff" framing was wrong on the premise and is superseded here.

Simplicity is already vendored and wired into consensus (Taproot leaf version `0xbe`, the `0xbe` dispatch, `SCRIPT_VERIFY_SIMPLICITY`) but gated off (`DEPLOYMENT_SIMPLICITY = NEVER_ACTIVE`). Taproot is always-active, so the substrate needs no prerequisite; activation is a one-flag flip that rides the planned testnet re-genesis, never a live-chain soft fork.

1. The core same-chain covenant offer

A maker locks N units of explicit asset A in one Taproot UTXO. The order IS the coin. Two spend branches:

- **FILL leaf** (Simplicity `0xbe`, permissionless, no signature): the UTXO is spendable by ANYONE iff the spending tx pays the maker the agreed counter-value of asset B. The leaf introspects a pinned output and asserts, all against explicit (non-blinded) values: the output's asset id equals the committed asset B, its amount is at least the required amount, and its scriptPubKey hash equals the committed maker payout script. `unwrap_right` on the confidential-value sum type hard-aborts if the taker tries to pay in a blinded output the covenant cannot read.
- **REFUND leaf**: an absolute-CLTV path returning A to the maker after `EXPIRY`, authorized by the maker's signature.

Taproot internal key: `P_maker` gives the maker a key-path cooperative-cancel/relist at any time (their own coin), at the cost of a fast-withdraw grief vector against a taker mid-fill; a NUMS internal key makes the offer purely covenant-governed with reclaim only via the CLTV refund. The wallet default is discussed in section 6.

Every order parameter (asset A id, asset B id, rate, maker payout script, min-lot, expiry) is a compile-

time constant baked into the tapleaf, so it is committed inside the Taproot output key. A taker cannot alter the terms; they can only satisfy them.

2. Partial fills (recursive covenant, no state carry)

A taker fills `filled` units (`min_lot <= filled <= locked`) iff the tx (a) pays the maker `ceil(filled * num / den)` of asset B (ceil rounds in the maker's favour), AND (b) returns the remainder `locked - filled` of asset A to a fresh output whose scriptPubKey EQUALS the covenant input's own scriptPubKey. That self-replication (the `last_will.simf` pattern: compare the whole spent scriptPubKey to the change output's scriptPubKey, tweak included) re-commits the ENTIRE parameter set for free.

Key result: the order's mutable size lives in the UTXO value, not in the program, and every constant is in the tapleaf, so the SAME program governs every remainder at every size. Therefore partial fills need NO `u256` state commitment at all: scriptPubKey equality plus the carried amount is sufficient. This is simpler than the assessment's sketched one-`u256`-carry.

Output-aliasing (two covenant inputs both crediting one shared maker-payment output, so one payment settles two orders) is the sharp edge. It is defeated structurally by a fixed input-bound output map: the covenant input at consensus index `k` must credit its maker payment and remainder at output indices derived from `k` (e.g. `2k, 2k+1`), so no two inputs can reference the same output. This also fixes the single-fill component's cross-offer aliasing bug (a maker's own price ladder sharing a reusable payout script). `filled >= min_lot` and `change == 0 OR change >= min_lot` floor both sides against dust-mint grieving.

Open arithmetic point: `filled * num` can exceed 64 bits for large orders; the covenant asserts the high limb is zero (bounding `num * max_size < 2^64`) or must route ceil-div through the `u256` multiply/divmod jets. This must be pinned before high-value use.

3. The cross-chain Sequentia-leg covenant

The Sequentia (asset) leg of a BTC to asset swap becomes a funded `0xbe` UTXO, mirroring today's SEQ HTLC P2SH: CLAIM = reveal a preimage `p` with `sha256(p) == H` AND pay asset A / amount A to the claimant's committed script at a pinned index; REFUND = CLTV to the funder after `T_seq`. Single-fill, all-or-nothing, so no recursion or state carry. The secret holder is always whoever claims the SEQ leg (the taker in forward BTC to asset, the maker in reverse asset to BTC); that claim reveals `s` on the Sequentia chain, which the counterparty reads to claim the Bitcoin leg. Only the Bitcoin leg stays an interactive HTLC/PTLC, because Bitcoin has no Simplicity.

Critical safety point: the covenant CANNOT introspect anchoring. Anchor height, `anchorstatus`, and quorum certification are not fields of the spending transaction, so the SEQ-leg covenant cannot enforce the anchor-ordering + quorum-cert gate (`VerifySeqLegSafe`). Making the SEQ leg unilaterally fillable therefore does NOT move that safety on-chain: the Bitcoin-leg claimant's WATCHER must still re-run `VerifySeqLegSafe` (anchor at or above the counter-leg height, `anchorstatus` ok, quorum

certified) on the reveal block BEFORE claiming the Bitcoin leg, and handle a reveal block that is later orphaned. The covenant funds and de-interactivizes the SEQ leg; it does not replace the watcher's anchor discipline. Consider requiring the claimant's signature in the CLAIM branch (filler-binding) so a third party who learns p cannot broadcast a pinned/high-fee reveal.

4. Order-book integration (relay demoted to discovery)

Add a settlement variant to the offer: `CovenantTerms covenant = 23` (after `same_chain=20`, `cross_chain=21`, `lightning=22`). For a covenant offer the existing offer fields change meaning: they are no longer a signed price PROMISE, they are a pointer plus the exact on-chain commitment a taker re-derives. Fields: `covenant_txid + covenant_vout` (the funded UTXO), `program_id + program_cmh` (the SimplicityHL template + its 32-byte commitment Merkle root), the Taproot internal key + merkle path, `rate_num/rate_den`, `maker_credit_spk`, `expiry_locktime`, `allow_partial + min_fill`, `credit_idx`. `maker_sig` stays but is NOT load-bearing for correctness (the chain is authoritative); it only authenticates the poster for relay rate-limiting and enables cancel/edit.

The property that demotes the relay: before filling, a taker verifies WITHOUT trusting the relay. It reads the UTXO on-chain (`gettxout`), reconstructs the tapleaf CMR from the versioned template + advertised constants, reconstructs the Taproot output key from the internal key + merkle path, and checks it equals the on-chain scriptPubKey. A relay can hide or lie about an offer but cannot fabricate depth, alter terms, or censor a taker who learns the outpoint. Front-running is bounded by the baked min-price (a reorged or sniped fill never underpays the maker); oversell is impossible because the order is the coin (first spend wins).

5. Must-fix findings (from the red-team)

1. **Hidden maker spend path.** A malicious maker can advertise a covenant UTXO whose Taproot internal key is a real key they control, letting them rug the coin out from under a taker. The taker MUST reconstruct the Taproot output from the advertised internal key + path and verify it matches the UTXO, and must treat a non-NUMS internal key as a maker-cancellable offer (or reject it). Trustless verification is not optional.
2. **Output-aliasing.** Fixed-index-per-covenant-input bijection (section 2), not a single global fixed index; per-order salt in the credited index.
3. **required_B overflow.** Bound order size so `num * size < 2^64`, or implement the u256 ceil-div. A wrong bound only self-harms the maker, but it must be handled.
4. **Cross-chain anchor safety stays a watcher discipline** (section 3), not covenant-enforced.
5. **MEV / sniping is an unpriced free option.** The min-price floor means it is never theft, only a race; optional mitigations are constraining the taker's asset-A receipt to a taker-committed key (reintroduces mild interactivity) or a client-side intent-to-fill backoff. Left permissionless in v1, surfaced honestly.
6. **Template registry.** `program_id` to SimplicityHL source must be distributed and version-pinned

so every taker reconstructs the CMR from a trusted template, not the relay's word.

6. The two tiers, and defaults

The covenant tier COMPLEMENTS, never replaces, the existing interactive tier. Covenant offers are funded, non-interactive, transparent, oversell-proof, relay-minimized; the interactive tier is unfunded signed intents with co-sign/HTLC settlement and supports the opt-in confidential path. A maker chooses per offer. Wallet defaults: prefer the covenant tier for transparent price-discovery markets once activated; keep the interactive tier for confidential or Bitcoin-leg-heavy flow. For the same-chain covenant offer, default the Taproot internal key to `P_maker` (instant cancel) with the CLTV refund as backstop; expose a NUMS "no-cancel, purely on-chain" option for makers who want it.

7. Activation

Flip `DEPLOYMENT_SIMPLICITY` to always-active (fresh testnet / re-genesis) or a dated schedule; fold it into the planned testnet re-genesis as a consensus change. No other consensus prerequisite (Taproot is already active). The covenant tier is opt-in and experimental until the Simplicity stack has an assurance bar beyond the Coq + internal review it has today.

8. Next: the regtest experiment

Smallest proof of the mechanics, on regtest with `DEPLOYMENT_SIMPLICITY = ALWAYS_ACTIVE`:

1. Write the single-fill same-chain covenant offer in SimplicityHL (section 1), modeled on `htlc.simf` (hashlock/CLTV shape) and `last_will.simf` (scriptPubKey self-comparison). Lock N of asset A; prove a taker fills unilaterally by paying asset B to the maker script, and that a wrong-asset, underpay, or blinded-output fill is REJECTED by consensus.
2. Extend to the partial-fill recursive covenant (section 2): prove a partial fill pays the maker pro-rata and returns the remainder to the same covenant, and that output-aliasing and dust-mint attempts are rejected.
3. Then the cross-chain SEQ-leg covenant (section 3) against a regtest Bitcoin HTLC leg, exercising the reveal path and confirming the watcher re-runs `VerifySeqLegSafe` before the Bitcoin claim.

This produces the SimplicityHL artifacts a covenant tier would build on, validates the activation path against our own tree, and does not touch the Phase-1 critical path.